

Lab03

AUTHOR

Giulia Marcellos

```
library(readr)
library(tidyverse)
```

— Attaching core tidyverse packages — tidyverse 2.0.0 —

```
✓ dplyr      1.2.1    ✓ purrr      1.2.2
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2     4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
```

— Conflicts — tidyverse_conflicts() —

✗ dplyr::filter() masks stats::filter()

✗ dplyr::lag() masks stats::lag()

ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

```
library(dplyr)
```

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airports <- read_csv("airports.csv", col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE))
```

Rows: 322 Columns: 5

— Column specification —

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

ℹ Use `spec()` to retrieve the full column specification for this data.

ℹ Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
head(airports)
```

A tibble: 6 × 5

	IATA_CODE	CITY	STATE	LATITUDE	LONGITUDE
	<chr>	<chr>	<chr>	<dbl>	<dbl>
1	ABE	Allentown	PA	40.7	-75.4
2	ABI	Abilene	TX	32.4	-99.7
3	ABQ	Albuquerque	NM	35.0	-107.
4	ABR	Aberdeen	SD	45.4	-98.4
5	ABY	Albany	GA	31.5	-84.2
6	ACK	Nantucket	MA	41.3	-70.1

Para o arquivo de vôos: a) Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY; Leia apenas 1 milhão de linhas por vez;

```
getStats <- function(x, pos) x

flights <- read_csv_chunked("flights.csv", callback = DataFrameCallback$new(getStats), chunk_s
```

Remova vôos em que o aeroporto de destino comece com a letra 1;

```
flights <- flights %>% filter(!is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY))
```

Remova registros em que existam pelo menos uma coluna faltante;

```
flights <- flights %>% drop_na()
```

Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;

```
getStats <- function(x, pos) x %>% filter(!startsWith(DESTINATION_AIRPORT, "1")) %>% drop_na()

stats <- read_csv_chunked("flights.csv", callback = DataFrameCallback$new(getStats), chunk_siz
```

Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
atraso <- stats %>% group_by(DESTINATION_AIRPORT) %>% summarise(n = sum(n), soma = sum(soma),
atraso
```

A tibble: 322 × 4

	DESTINATION_AIRPORT	n	soma	media
	<chr>	<int>	<dbl>	<dbl>
1	ABE	2220	12874	5.80
2	ABI	2224	9077	4.08
3	ABQ	18953	106415	5.61
4	ABR	657	-2227	-3.39
5	ABY	864	7501	8.68
6	ACK	487	2393	4.91
7	ACT	1541	16392	10.6
8	ACV	1267	12259	9.68
9	ACY	3530	41744	11.8
10	ADK	89	-284	-3.19

i 312 more rows

2) Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa: Identifique a coluna que é a chave na tabela flights; Identifique a coluna que é a chave na tabela airports; Quais são os aeroportos que estão listados em flights, mas

estão ausentes em airports? Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?

```
flights %>% anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>% distinct(DEST
```

```
# A tibble: 307 × 1
  DESTINATION_AIRPORT
  <chr>
1 11298
2 13487
3 13303
4 11057
5 13930
6 10693
7 14747
8 12266
9 12478
10 14057
# i 297 more rows
```

```
flights1 <- flights %>% left_join(airports %>% select(IATA_CODE, CITY, STATE), by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
flights1
```

```
# A tibble: 5,714,008 × 4
  DESTINATION_AIRPORT ARRIVAL_DELAY CITY STATE
  <chr>                <dbl> <chr>   <chr>
1 SEA                 -22 Seattle WA
2 PBI                 -9 West Palm Beach FL
3 CLT                  5 Charlotte NC
4 MIA                 -9 Miami FL
5 ANC                -21 Anchorage AK
6 MSP                  8 Minneapolis MN
7 MSP                -17 Minneapolis MN
8 CLT                -10 Charlotte NC
9 DFW                -13 Dallas-Fort Worth TX
10 ATL                -15 Atlanta GA
# i 5,713,998 more rows
```

3) Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aero_estado <- airports %>% group_by(STATE) %>% summarise(n_aeroportos = n(), .groups = "drop")
aero_estado
```

```
# A tibble: 54 × 2
  STATE n_aeroportos
  <chr>   <int>
1 TX         24
2 CA         22
```

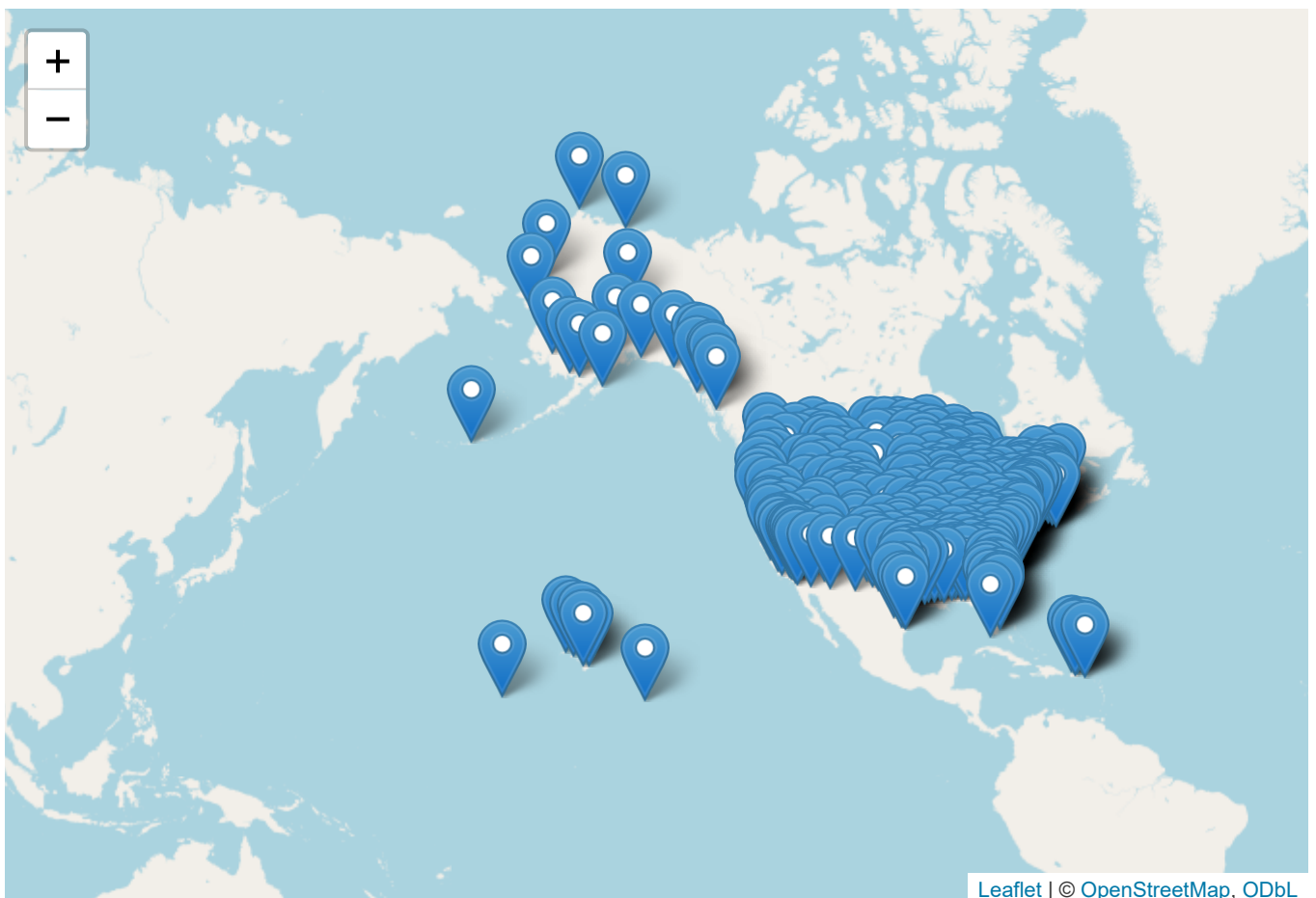
3	AK	19
4	FL	17
5	MI	15
6	NY	14
7	CO	10
8	MN	8
9	MT	8
10	NC	8

i 44 more rows

4) Apresente um mapa representando todos os atrasos observados por aeroporto.

Carregue o pacote leaflet; Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico; Armazene o grafico em b) numa variável chamada this; Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
library(leaflet)
mapa <- airports %>% filter(!is.na(LATITUDE), !is.na(LONGITUDE))
this <- leaflet(mapa) %>% addTiles() %>% addMarkers(lng = ~LONGITUDE, lat = ~LATITUDE)
this
```



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-31 20:40:31 -03"
```

